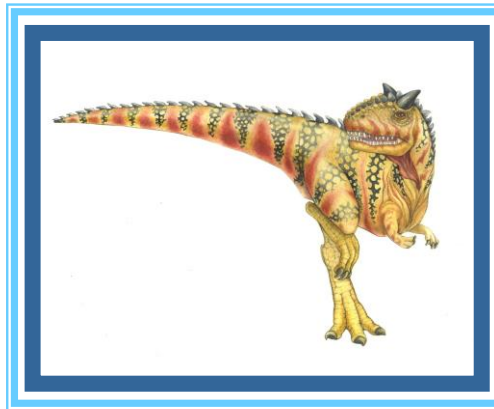
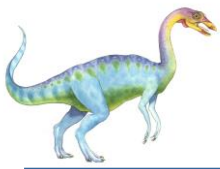


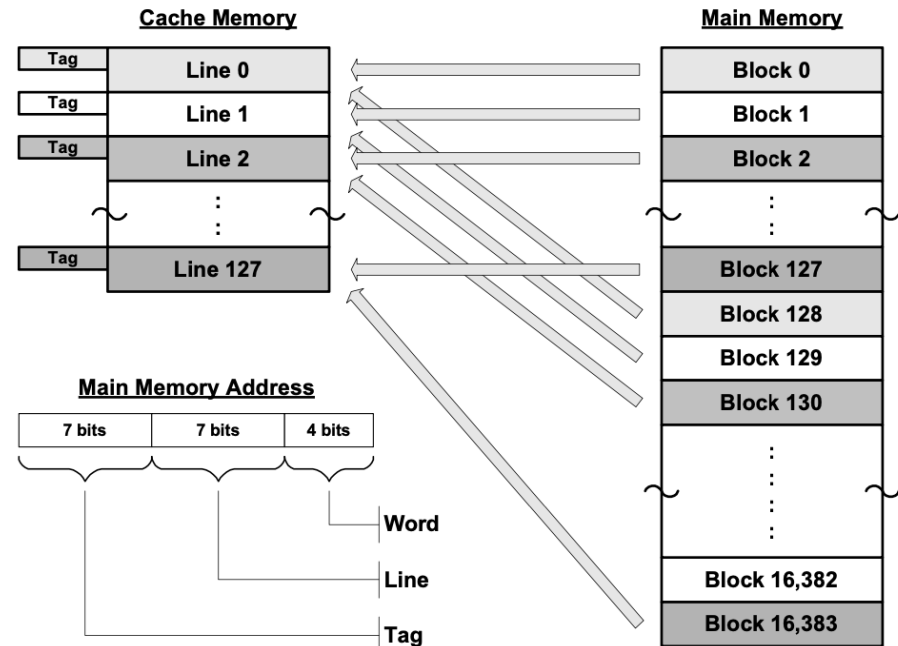
Chapter 12: Cache

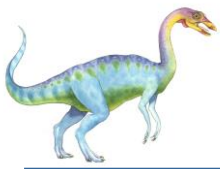




Direct Mapping

- ❑ Which line to store the information?
 - ❑ Line ID = Block ID mod number of lines
- ❑ How many lines are there in a cache?
 - ❑ number of lines = cache size / words per line
 - ❑ E.g., cache size 2048 words, 16 words per line, there are 128 lines
 - ❑ Line bits = $\log_2(\text{number of lines})$
 - ❑ Cache size: $2^{(\text{line bits} + \text{word bits})}$
- ❑ How many tags are there?
 - ❑ number of tags = number of blocks / number of lines (or memory size / cache size)
 - ❑ Tag bits = $\log_2(\text{number of tags})$
 - ❑ Memory size: $2^{(\text{tag bits} + \text{line bits} + \text{word bits})}$

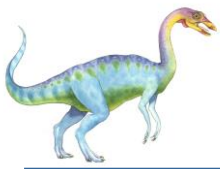




Direct Mapping cont.

- To search for a word in the cache,
 - Determine what line to look in – easy; just index into the cache using the line bits of the address.
 - Compare the tag bits (leading seven bits) of the address with the tag of the line. If it matches, the block is in the cache.
 - Select the desired word from the line using the word bits.
- Advantages:
 - Fast lookup (only one comparison needed).
 - Cheap hardware (no associative comparison).
 - Easy to decide where it is or where to place it.

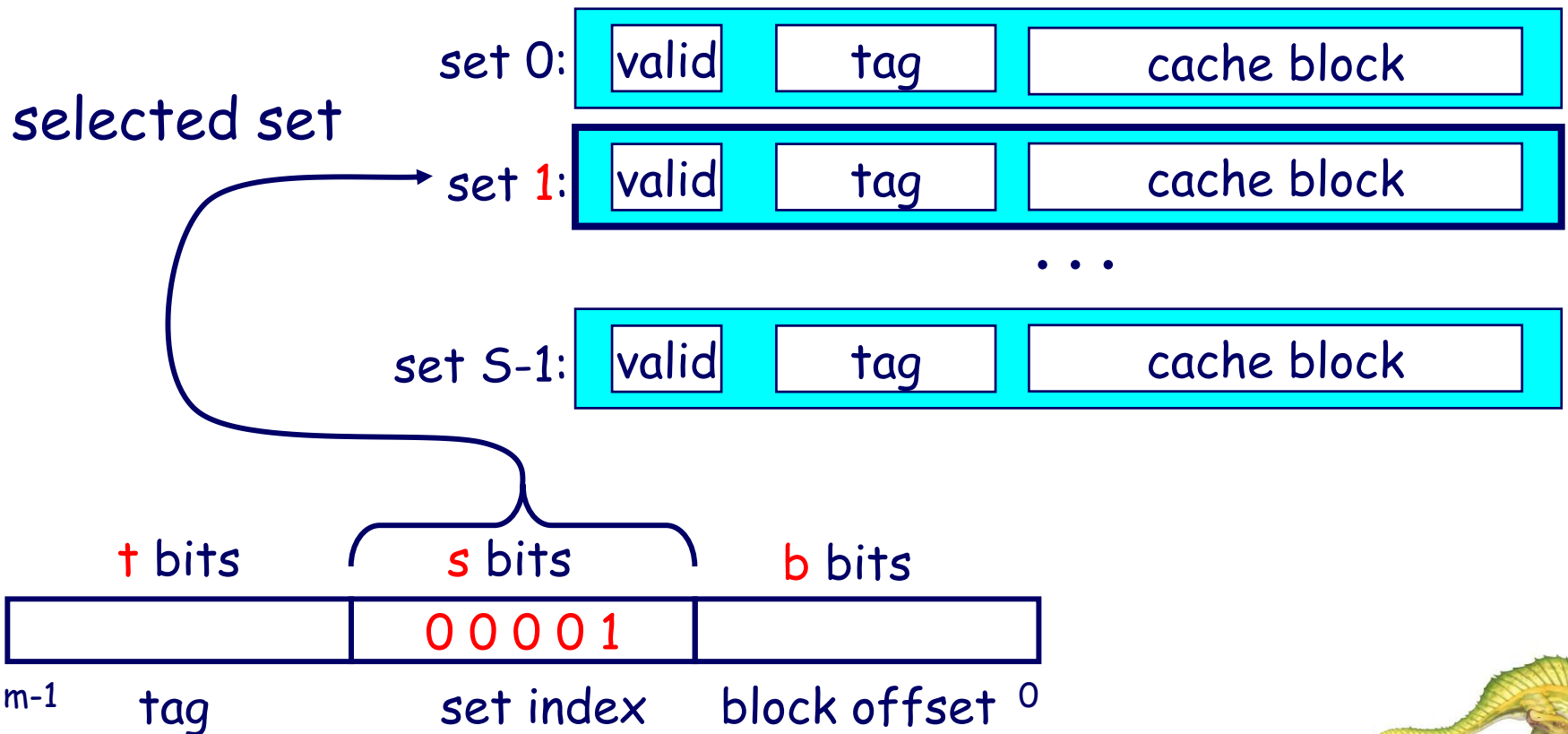


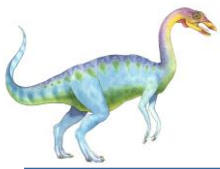


Accessing Direct-Mapped Caches

Set selection

- Use the set index bits to determine the set of interest.

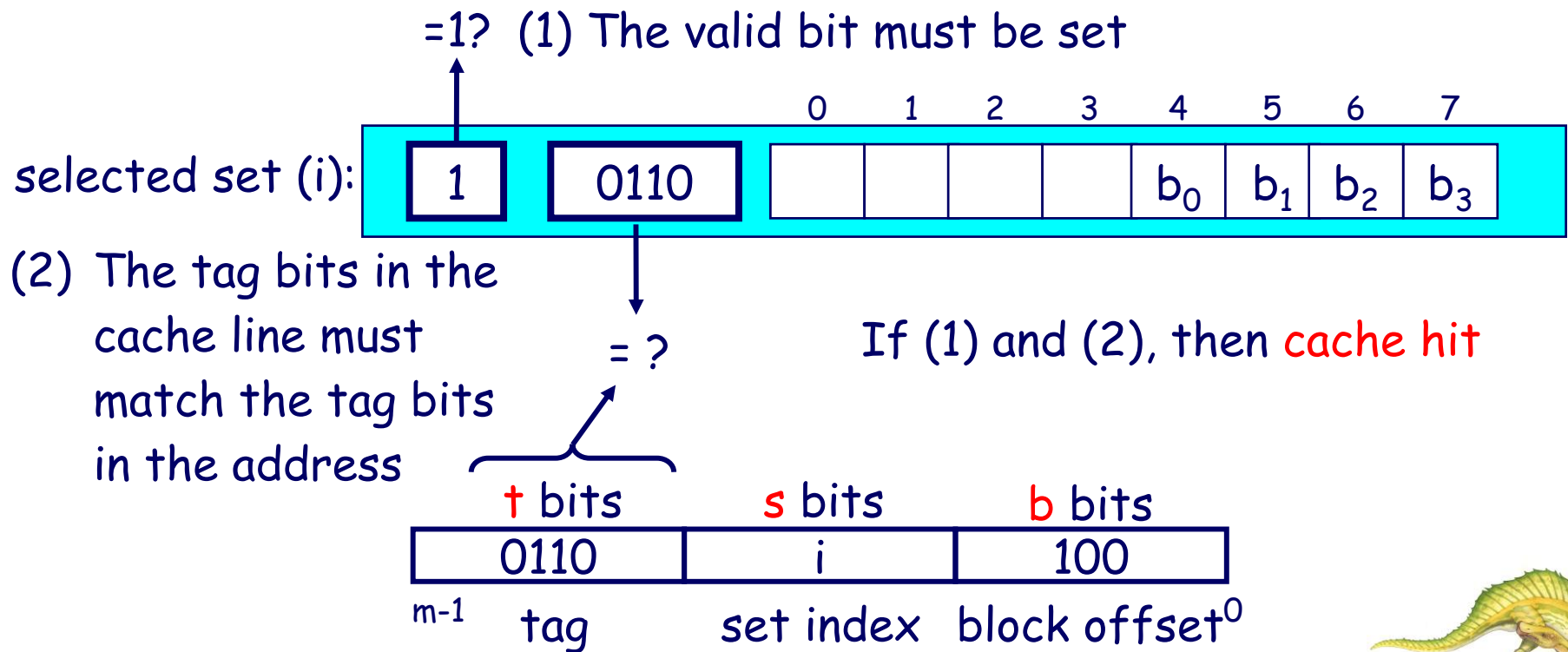


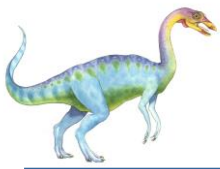


Accessing Direct-Mapped Caches

Line matching and word selection

- **Line matching:** Find a valid line in the selected set with a matching tag
- **Word selection:** Then extract the word

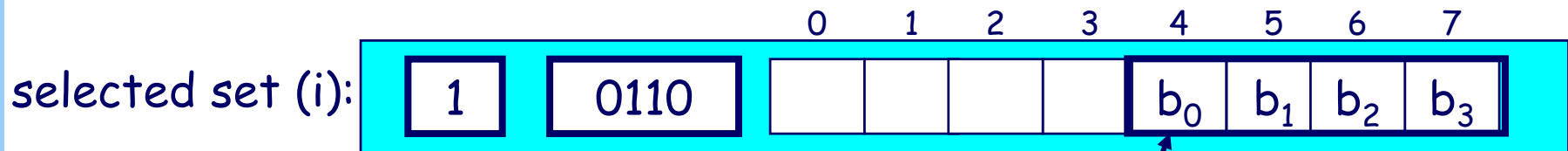




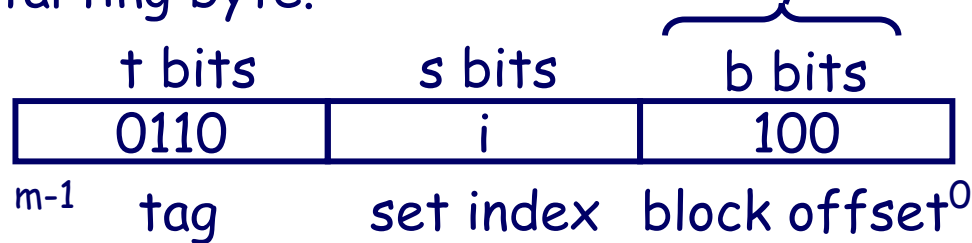
Accessing Direct-Mapped Caches

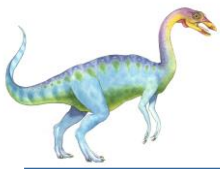
Line matching and word selection

- **Line matching:** Find a valid line in the selected set with a matching tag
- **Word selection:** Then extract the word



(3) If cache hit,
block offset selects
starting byte.





Direct-Mapped Cache Simulation

t=1 s=2 b=1

x	xx	x
---	----	---

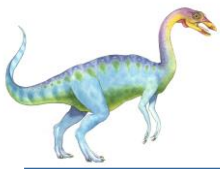
M=16 byte addresses, B=2 bytes/block,
S=4 sets, E=1 entry/set

Address trace (reads):

0	[0000 ₂],	miss
1	[0001 ₂],	hit
7	[0111 ₂],	miss
8	[1000 ₂],	miss
0	[0000 ₂]	miss

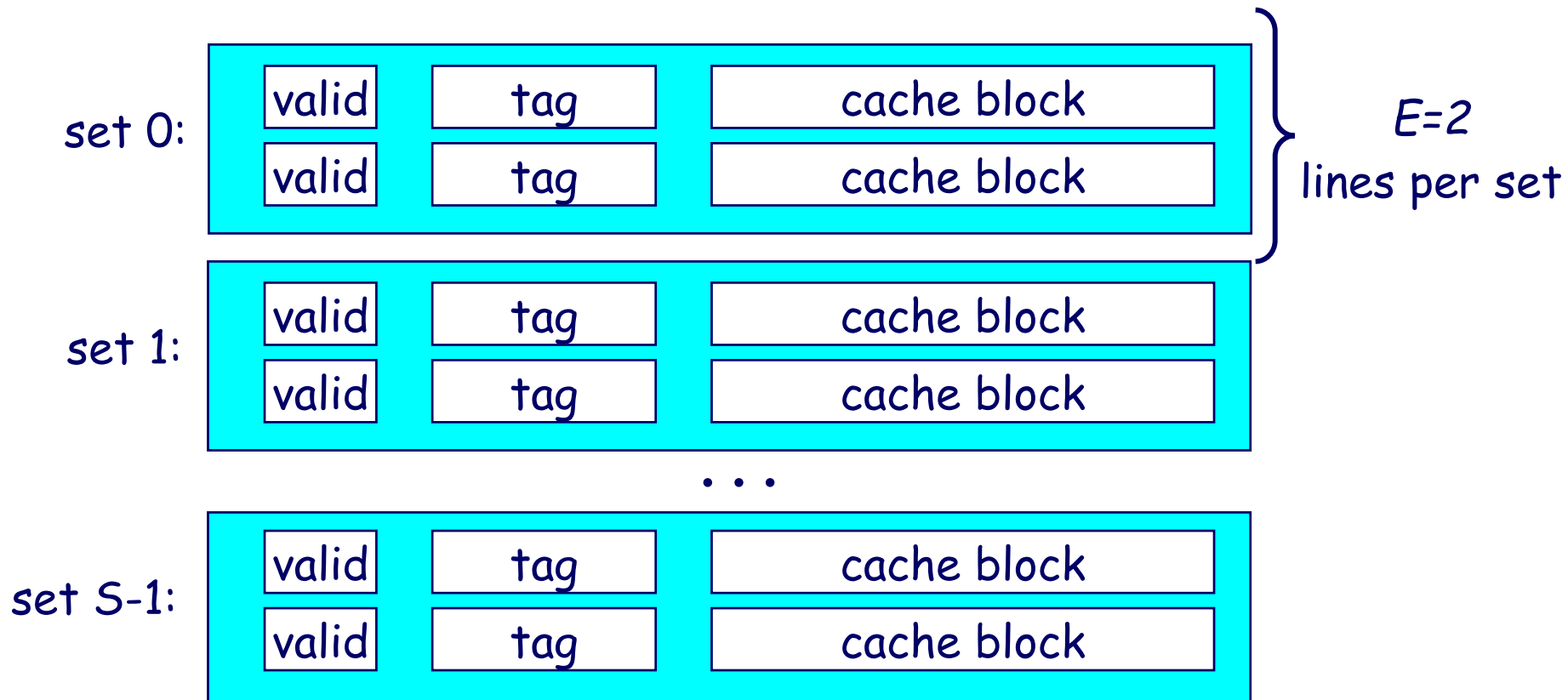
set	v	tag	data
00	1	0	M[0-1]
01			
10			
11	1	0	M[6-7]





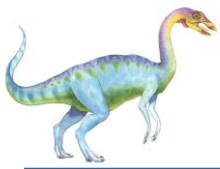
Example: Set Associative Cache

Characterized by more than one line per set



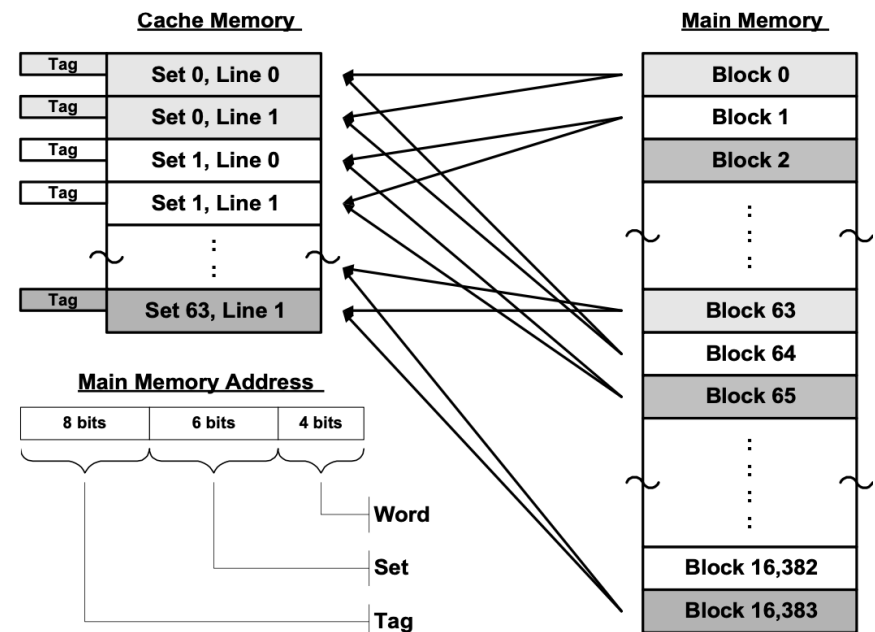
E-way associative cache





Set-associative Cache

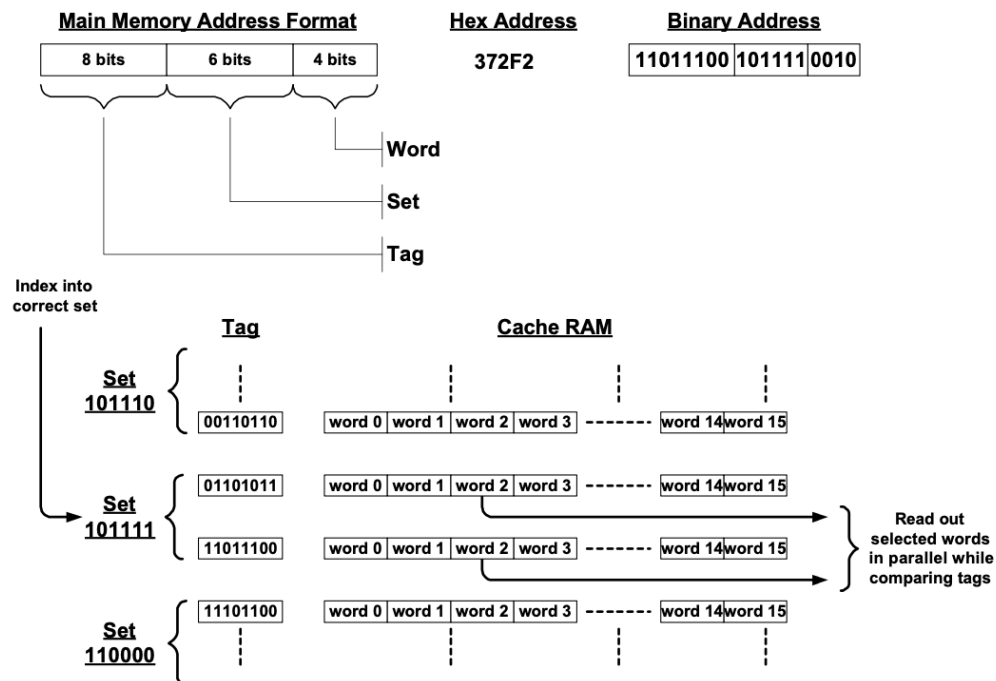
- Give away line/set bits to tag bits
 - 2-way set associative cache gives away 1 line/set bit to tag bit
- Which set to store the information?
 - Set ID = Block ID mod number of sets
- How many sets are there in a cache?
 - number of sets = (cache size / words per line) / lines per set
 - E.g., cache size 2048 words, 16 words per line, 2-way associative cache has 64 sets
 - Set bits = $\log_2(\text{number of sets})$
 - Cache size: $2^{(\text{set bits} + \text{word bits})} \times \text{lines per set}$
- How many tags are there?
 - number of tags = number of blocks / number of sets
 - Tag bits = $\log_2(\text{number of tags})$
 - Memory size: $2^{(\text{tag bits} + \text{set bits} + \text{word bits})}$





Accessing in Set-associative Cache

- Index into the set using the set bits of the address ($i \bmod s$).
- Simultaneously compare the tag bits (leading 8 bits) of the address with the tag of all lines in the set. If it matches any one, the desired block is in the cache.
- Concurrent with comparing the tag bits, begin reading out the selected data words of all lines in the set (using the word bits) so the data words will be available at the end of the compare cycle.
- If a tag match is found, get the data word from the selected line to the cache-output buffer and on to the processor. Discard the other words.





2-Way Associative Cache Simulation

M=16 byte addresses, B=2 bytes/block,
S=2 sets, E=2 entry/set

t=2 s=1 b=1

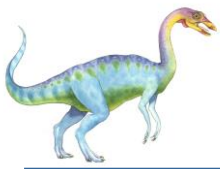
xx	x	x
----	---	---

Address trace (reads):

0	[0000 ₂]	miss
1	[0001 ₂]	hit
7	[0111 ₂]	miss
8	[1000 ₂]	miss
0	[0000 ₂]	hit

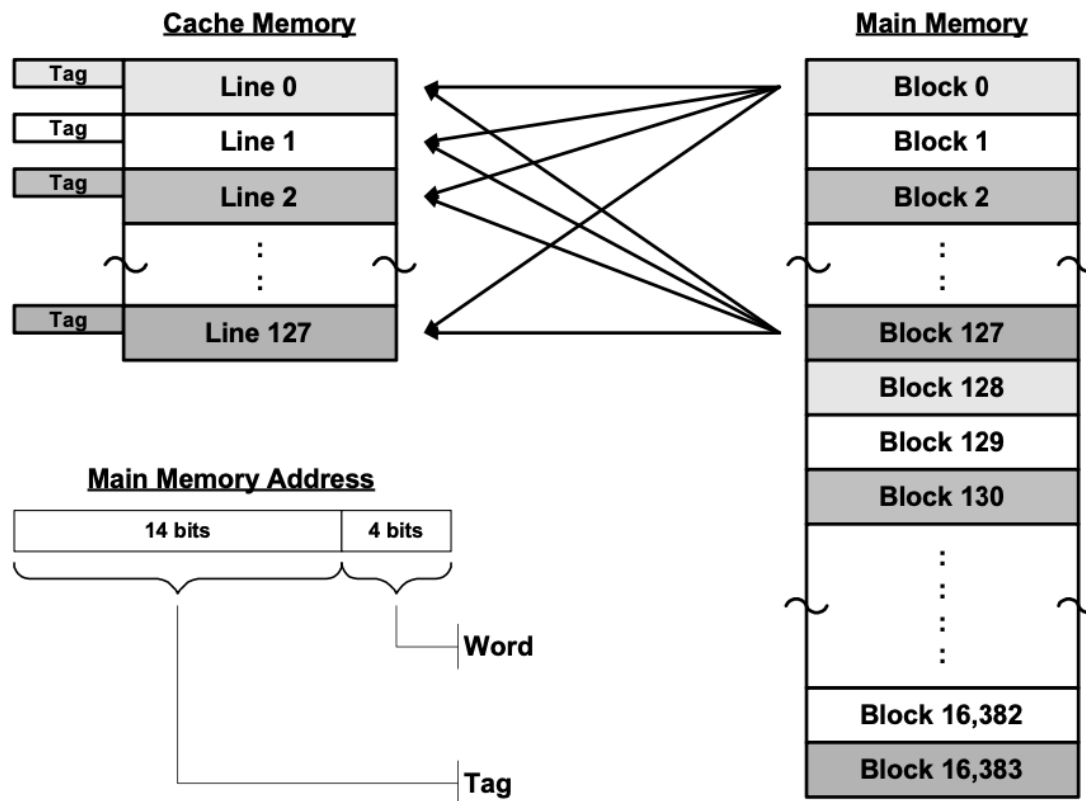
set	v	tag	data
0	1	00	M[0-1]
	1	10	M[8-9]
1	1	01	M[6-7]
	0		





Fully-associative Cache

- Give away all set bits to tag bits
- Less collision, but slow look up





In Class Practice

The following cache represents a 2-way set associative cache, i.e., there are two lines per set. Notice that the set ID values start at 01101101_2 and increment every other row. This is meant to imply that you are looking at a group of lines/sets toward the middle of the cache and not the entire cache. There are 14 bits for the tag, 8 bits for the set id, and 2 bits for the word id. Answer the following 3 questions based on this cache.

Tag (binary values)	Set ID (binary values)	Word within block			
		00	01	10	11
10100001001001	01101101	00_{16}	61_{16}	$C2_{16}$	23_{16}
11100001100100	01101101	10_{16}	71_{16}	$D2_{16}$	33_{16}
11001011010110	01101110	20_{16}	81_{16}	$E2_{16}$	43_{16}
11100101101011	01101110	30_{16}	91_{16}	$F2_{16}$	53_{16}
11110110110100	01101111	40_{16}	$A1_{16}$	02_{16}	63_{16}
10100111010101	01101111	50_{16}	$B1_{16}$	12_{16}	73_{16}
10101010111110	01110000	84_{16}	$E5_{16}$	46_{16}	$A7_{16}$
10101010010011	01110000	94_{16}	$F5_{16}$	56_{16}	$B7_{16}$
01110001001000	01110001	$A4_{16}$	$A5_{16}$	66_{16}	$C7_{16}$
00001101101101	01110001	$B4_{16}$	15_{16}	76_{16}	$D7_{16}$
01011010010010	01110010	$C4_{16}$	25_{16}	86_{16}	$E7_{16}$
10101111001011	01110010	$D4_{16}$	35_{16}	96_{16}	$F7_{16}$

1. What is the data for the address of 7121C5 (hex)? **A5**
2. What is the total memory size? **16MB**
3. What is the total cache size? **2KB**



End of Chapter 12

